TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 7: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 16 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

A1. State the closest-pair-of-points problem precisely. What is the running time of the brute-force algorithm, and why is it $\Theta(n^2)$?

A2. In the divide-and-conquer closest-pair algorithm, after recursing on the left and right halves we obtain $\delta = \min(\delta_L, \delta_R)$. Explain why any crossing pair (one point in each half) that is closer than δ must lie entirely within the vertical strip of width 2δ centred on the dividing line.

A3. Prove the "sparsity" lemma: when the strip points are sorted by y-coordinate, two points within distance δ of each other are within a constant number of positions in that order. Use the $\delta/2$-box argument and explain why no box contains two points.

A4. Why does the closest-pair algorithm pre-sort the points by y-coordinate *once* at the top level, and pass the y-sorted sublists down through the recursion, rather than re-sorting the strip by y inside each call? What would re-sorting do to the recurrence?

A5. Write and solve the recurrence for the divide-and-conquer closest-pair algorithm. Identify the cost of the combine (strip) step and explain why it is linear.

A6. Derive the naive $\Theta(n^2)$ recurrence for integer multiplication by splitting each n-digit number into two halves. Exactly which four products appear, and why does the master theorem give $\Theta(n^2)$?

A7. State Karatsuba's identity: how are the three products z_0, z_1, z_2 defined, and why does z_1 equal the desired middle coefficient $x_1y_0 + x_0y_1$? Write the resulting recurrence and solve it.

A8. Strassen's algorithm multiplies two $n \times n$ matrices using 7 recursive multiplications of $(n/2) \times (n/2)$ blocks instead of 8. Write both recurrences (8-way and 7-way) with an $O(n^2)$ combine step and solve each. For which n does Strassen perform strictly fewer scalar multiplications?

A9. Explain why multiplying two polynomials given by their coefficient vectors is exactly the convolution of those vectors. Give the formula for the k-th coefficient of the product.

A10. A degree-$(n-1)$ polynomial can be represented by n coefficients or by its values at n distinct points. Explain why multiplication is $O(n)$ in the value representation but $\Theta(n^2)$ in the coefficient representation, and why this motivates the FFT.

A11. Define the n-th roots of unity ω_n^k. State and prove the squaring (halving) property $(\omega_n^k)^2 = \omega_{n/2}^k$ and the cancellation property $\omega_n^{k+n/2} = -\omega_n^k$.

A12. Derive the even/odd splitting identity $A(x) = A_{\text{even}}(x^2) + x\,A_{\text{odd}}(x^2)$ and explain how, combined with the two root-of-unity properties, it lets the FFT compute the length-n DFT from two length-$(n/2)$ DFTs in $O(n)$ extra work. Write the resulting recurrence.

A13. Show that the inverse DFT has the same form as the forward DFT with ω_n replaced by ω_n^{-1} and an extra factor of $1/n$. Prove the key orthogonality identity $\frac{1}{n}\sum_{k=0}^{n-1} \omega_n^{k(j-j')} = [\,j = j'\,]$.

A14. Describe the complete $O(n \log n)$ algorithm for multiplying two polynomials via the FFT (padding, forward transforms, pointwise product, inverse transform, rounding). Prove its correctness, i.e. that $\hat{C}[k] = C(\omega^k)$ for the true product $C = AB$.

A15. Explain how counting the number of contiguous subarrays whose sum lies in $[\ell, u]$ reduces to a problem on the prefix-sum array, and how a mergesort with a two-pointer count during the merge solves it in $O(n \log n)$.

A16. Describe the $O(\log(m+n))$ algorithm for the median of two sorted arrays. What is the partition invariant that the binary search maintains, and why does it guarantee the median is found?

Part B: Quiz

B1. (Multiple choice) The divide-and-conquer closest-pair algorithm runs in:

(a) $\Theta(n^2)$

(b) $\Theta(n \log n)$

(c) $\Theta(n)$

(d) $\Theta(n \log^2 n)$

B2. (True/False) In the closest-pair strip, each point need only be compared with a constant number of its neighbours in y-sorted order (e.g. the next 7), regardless of n.

B3. (Multiple choice) Karatsuba's algorithm multiplies two n-digit integers in time:

(a) $\Theta(n)$

(b) $\Theta(n \log n)$

(c) $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$

(d) $\Theta(n^2)$

B4. (Short answer) How many recursive multiplications does Karatsuba use per level, and how many does the naive split use?

B5. (Multiple choice) Strassen's matrix multiplication has running time:

(a) $\Theta(n^2)$

(b) $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$

(c) $\Theta(n^3)$

(d) $\Theta(n^2 \log n)$

B6. (True/False) Multiplying two polynomials given as coefficient vectors is the same as convolving those vectors.

B7. (Short answer) What is $\omega_n^{k+n/2}$ in terms of ω_n^k, and what is this property called?

B8. (Multiple choice) The recursive radix-2 FFT computes the DFT of a length-n vector (n a power of two) in time:

(a) $\Theta(n^2)$

(b) $\Theta(n)$

(c) $\Theta(n \log n)$

(d) $\Theta(\log n)$

B9. (True/False) The inverse DFT can be computed by the same FFT routine using the conjugate root ω_n^{-1} and dividing the result by n.

B10. (Short answer) In fast modular exponentiation (square-and-multiply), roughly how many modular multiplications are needed to compute $a^e \bmod m$?

Part C: Solutions

Part A

A1. **Problem.** Given n points $p_1, \ldots, p_n$ in the plane, find the pair minimizing the Euclidean distance $d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. The brute-force algorithm computes the distance for all $\binom{n}{2} = \frac{n(n-1)}{2}$ pairs and keeps the minimum; this is $\Theta(n^2)$ because it does a constant amount of work for each of $\Theta(n^2)$ pairs.

A2. Suppose a crossing pair (p, q) has $p \in L$, $q \in R$, and $d(p, q) < \delta$. Since the Euclidean distance is at least the difference in any single coordinate, $|x_p - x_q| \le d(p, q) < \delta$. The dividing line $x = x^*$ lies between x_p (on the left) and x_q (on the right), so $|x_p - x^*| \le |x_p - x_q| < \delta$ and similarly $|x_q - x^*| < \delta$. Hence both points are within horizontal distance δ of the line, i.e. inside the strip.

A3. Partition the strip (width 2δ) into a grid of square boxes of side $\delta/2$. Each box has diameter $\frac{\delta}{2}\sqrt{2} = \frac{\delta}{\sqrt{2}} < \delta$. No box can contain two points: two points in the same box would be at distance $< \delta$, but every pair of points lying on the same side of the dividing line is at distance $\ge \delta$ (guaranteed by the recursive calls), and a box of width $\delta/2$ small enough to straddle the line still has diameter $< \delta$, so two points in it would contradict δ being the current minimum. Now if two strip points are within distance δ, their y-values differ by $\le \delta$, so the second lies within a fixed-size block of boxes around the first (the strip is 4 columns wide, and a height of δ above/below spans a constant number of rows). Since each box holds at most one point, only a constant number (≤ 15, and after a more careful count ≤ 7 in y-order) of points can be that close. Hence comparing each point with its next 7 (or 15) neighbours in y-order suffices.

A4. Pre-sorting by y once costs $O(n \log n)$ at the top level only. Each recursive call then produces its left/right y-sorted sublists by a single linear scan of its already y-sorted input (a partition, not a sort), keeping the per-call combine cost $O(n)$. If instead we re-sorted the strip by y inside every call, each call would cost $O(n \log n)$, giving the recurrence $T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$ – a $\log n$ factor worse. Pre-sorting preserves the $O(n)$ combine and hence the $O(n \log n)$ total.

A5. The recurrence is $T(n) = 2\,T(n/2) + O(n)$. The two recursive calls handle the halves; the $O(n)$ combine splits P_y in linear time and scans the strip doing a constant number of distance comparisons per strip point (by A3), which is $O(|S|) = O(n)$. By the master theorem ($a = 2$, $b = 2$, $d = 1$, $a = b^d$), $T(n) = \Theta(n \log n)$. (The initial sort is a one-time $O(n \log n)$ cost that does not change the bound.)

A6. Write $x = x_1 B^{n/2} + x_0$ and $y = y_1 B^{n/2} + y_0$. Then $xy = x_1 y_1 B^n + (x_1 y_0 + x_0 y_1) B^{n/2} + x_0 y_0$, requiring the four products $x_1 y_1$, $x_1 y_0$, $x_0 y_1$, $x_0 y_0$, each a multiplication of $(n/2)$-digit numbers, plus $O(n)$ additions and shifts. So $T(n) = 4\,T(n/2) + O(n)$; with $a = 4$, $b = 2$, $d = 1$, we have $a = 4 > b^d = 2$, so $T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$.

A7. Define $z_2 = x_1 y_1$, $z_0 = x_0 y_0$, and $z_1 = (x_0 + x_1)(y_0 + y_1) - z_2 - z_0$. Expanding, $(x_0 + x_1)(y_0 + y_1) = x_0 y_0 + x_0 y_1 + x_1 y_0 + x_1 y_1 = z_0 + (x_0 y_1 + x_1 y_0) + z_2$, so subtracting z_0 and z_2 leaves $z_1 = x_0 y_1 + x_1 y_0$, exactly the middle coefficient. Then $xy = z_2 B^n + z_1 B^{n/2} + z_0$ uses only three recursive multiplications (z_0, z_2, and the product inside z_1), giving $T(n) = 3\,T(n/2) + O(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

A8. The standard recursive 2x2-block multiply does 8 block multiplications: $T(n) = 8\,T(n/2) + O(n^2)$, with $a = 8$, $b = 2$, $d = 2$, $a = 8 > b^d = 4$, so $T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$. Strassen uses 7: $T(n) = 7\,T(n/2) + O(n^2)$, with $a = 7 > b^d = 4$, so $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$. Counting only scalar multiplications with a 1×1 base case costing 1: the naive scheme uses $8^{\log_2 n} = n^3$ and Strassen uses $7^{\log_2 n} = n^{\log_2 7}$; Strassen is strictly fewer for every $n \geq 2$ (a power of two), since $7 < 8$ at each of the $\log_2 n \geq 1$ levels.

A9. If $A(x) = \sum_i a_i x^i$ and $B(x) = \sum_j b_j x^j$, then $A(x)B(x) = \sum_k \big(\sum_{i+j=k} a_i b_j\big) x^k$. So the k-th coefficient of the product is $c_k = \sum_{i+j=k} a_i b_j$, which is precisely the (linear) convolution of the coefficient vectors (a_i) and (b_j).

A10. In the value representation, if A and B are known at the same set of $\geq \deg A + \deg B + 1$ points, then (AB) at each point is just the product of the two values – $O(n)$ pointwise multiplications total. In the coefficient representation, computing the convolution directly is the double sum over all i, j, costing $\Theta(n^2)$. This asymmetry motivates the FFT: convert coefficients $\to$ values (evaluate), multiply pointwise in $O(n)$, then convert values $\to$ coefficients (interpolate). The FFT makes both conversions $O(n \log n)$, so the whole multiplication is $O(n \log n)$.

A11. $\omega_n^k = e^{2\pi i k/n}$ are the n equally spaced points on the unit circle with $z^n = 1$. *Squaring:* $(\omega_n^k)^2 = e^{2\pi i \cdot 2k/n} = e^{2\pi i k/(n/2)} = \omega_{n/2}^k$. *Cancellation:* $\omega_n^{k+n/2} = e^{2\pi i (k+n/2)/n} = e^{2\pi i k/n} \cdot e^{\pi i} = \omega_n^k \cdot (-1) = -\omega_n^k$.

A12. Separating even- and odd-indexed coefficients, $A(x) = \sum_j a_{2j} x^{2j} + \sum_j a_{2j+1} x^{2j+1} = A_{\text{even}}(x^2) + x\,A_{\text{odd}}(x^2)$, where $A_{\text{even}}, A_{\text{odd}}$ have degree $< n/2$. Evaluating at $x = \omega_n^k$: by the squaring property $(\omega_n^k)^2 = \omega_{n/2}^k$, so A_{even} and A_{odd} are evaluated at the $(n/2)$-th roots of unity – two DFTs of size $n/2$. By cancellation, for $k < n/2$, $A(\omega_n^k) = E[k] + \omega_n^k O[k]$ and $A(\omega_n^{k+n/2}) = E[k] - \omega_n^k O[k]$, so all n outputs are produced from the two half-size DFTs with $n/2$ butterflies, i.e. $O(n)$ extra work. The recurrence is $T(n) = 2\,T(n/2) + O(n) = O(n \log n)$.

A13. The forward DFT is multiplication by V with $V_{kj} = \omega_n^{kj}$. Claim: $(V^{-1})_{kj} = \frac{1}{n}\omega_n^{-kj}$. Compute $(V \cdot \frac{1}{n}\overline{V})_{jj'} = \frac{1}{n}\sum_k \omega_n^{jk}\omega_n^{-kj'} = \frac{1}{n}\sum_k \omega_n^{k(j-j')}$. If $j = j'$, every term is 1, giving $\frac{1}{n} \cdot n = 1$. If $j \neq j'$, set $r = \omega_n^{j-j'} \neq 1$; then $\sum_{k=0}^{n-1} r^k = \frac{r^n - 1}{r - 1} = 0$ because $r^n = 1$. Hence the product is the identity, proving $V^{-1} = \frac{1}{n}\overline{V}$: the inverse DFT is the same sum with ω_n replaced by ω_n^{-1}, scaled by $1/n$.

A14. **Algorithm.** Pad A, B with zeros to a common length m (a power of two) with $m \geq \deg A + \deg B + 1$. Compute $\hat{A} = \text{FFT}(A)$ and $\hat{B} = \text{FFT}(B)$ (values at the m-th roots of unity). Form $\hat{C}[k] = \hat{A}[k]\hat{B}[k]$ for all k. Compute $C = \text{IFFT}(\hat{C})$ and round to integers if the inputs were integer-valued. **Correctness:** $\hat{C}[k] = \hat{A}[k]\hat{B}[k] = A(\omega^k)B(\omega^k) = C(\omega^k)$, so $\hat{C}$ holds the values of the true product $C = AB$ at the m roots of unity. Since $\deg C = \deg A + \deg B < m$, these m values determine C uniquely, and the inverse FFT recovers its coefficients exactly. The cost is three FFTs ($O(m \log m)$) plus the $O(m)$ pointwise product, i.e. $O(n \log n)$.

A15. Let $S[0], \ldots, S[n]$ be the prefix sums, $S[0] = 0$, $S[t] = \sum_{i<t} \text{nums}[i]$. A subarray spanning indices (i, j) has sum $S[j] - S[i]$, so we must count pairs $i < j$ with $\ell \leq S[j] - S[i] \leq u$. Mergesort the array S. When merging two sorted halves (left indices i, right indices j, with all left positions before all right positions in the original order), for each left value $S[i]$ slide two pointers over the sorted right half to count the j with $S[j] - S[i] \in [\ell, u]$ – a $\ell \leq S[j] - S[i]$ lower pointer and a $S[j] - S[i] \leq u$ upper pointer, each advancing monotonically. This counts

all straddling pairs in $O(n)$ per merge level, and recursion handles the within-half pairs, for $O(n \log n)$ total.

A16. Assume WLOG a is the shorter array. Binary-search the number i of elements of a placed on the left side of a partition; set $j = \lceil (m+n)/2 \rceil - i$ so the left side holds exactly $\lceil (m+n)/2 \rceil$ elements. Using sentinels $a_{\text{left}} = a[i-1]$ (or $-\infty$), $a_{\text{right}} = a[i]$ (or $+\infty$), and likewise for b, the partition is *correct* when $a_{\text{left}} \le b_{\text{right}}$ and $b_{\text{left}} \le a_{\text{right}}$: this invariant says every element on the left is $\le$ every element on the right, so the boundary is the true median split. Then the median is $\max(a_{\text{left}}, b_{\text{left}})$ for odd total length, or the average of that and $\min(a_{\text{right}}, b_{\text{right}})$ for even length. If $a_{\text{left}} > b_{\text{right}}$ we move i left; otherwise we move i right. The search runs in $O(\log \min(m,n)) = O(\log(m+n))$.

Part B

B1. **(b)** $\Theta(n \log n)$**.** The recurrence is $T(n) = 2T(n/2) + O(n)$.

B2. **True.** The $\delta/2$-box / sparsity lemma shows only a constant number of strip points can be within δ of any given point in y-order.

B3. **(c)** $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$**.** From $T(n) = 3T(n/2) + O(n)$.

B4. Karatsuba uses **3** recursive multiplications per level; the naive split uses **4**.

B5. **(b)** $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$**.** From $T(n) = 7T(n/2) + O(n^2)$.

B6. **True.** The product coefficients are exactly the convolution $c_k = \sum_{i+j=k} a_i b_j$.

B7. $\omega_n^{k+n/2} = -\omega_n^k$; this is the **cancellation property** (a root and its antipode differ only in sign).

B8. **(c)** $\Theta(n \log n)$**.** From $T(n) = 2T(n/2) + O(n)$ with $n/2$ butterflies per level.

B9. **True.** $V^{-1} = \frac{1}{n}\overline{V}$, so running the FFT with ω_n^{-1} and dividing by n inverts the transform.

B10. About $\Theta(\log e)$ modular multiplications: one squaring per bit of e plus one extra multiply per set bit, so at most $2\lfloor \log_2 e \rfloor + 1$.